CHECKPOINT 2 - RPA UTILIZANDO PYTHON

Automação e Integração com a API do Spotify

EQUIPE

Integrante	RM
Tomáz Versolato Carballo	551417
Rony Ken Nagai	551549
Aksel Viktor Caminha Rae	99011
Ian Xavier Kuraoka	98860

DISCIPLINA

RPA - Checkpoint 2

FIAP • 2026

SUMÁRIO

Objetivo 2
1. Solução adotada 2
2. Tecnologias utilizadas 2
3. Opções de playlists 2
4. Fluxo da automação 2
5. Endpoints da Spotify Web API utilizados 3
6. Permissões solicitadas 3
7. Preparação para a validação em aula 3
8. Tratamento de falhas 3
9. Estrutura do projeto 4
10. Atendimento aos requisitos do CP2 4
11. Conclusão 4
Referências técnicas 4

Objetivo

Desenvolver uma automação em Python capaz de resolver uma tarefa real por meio de integração REST. A solução permite selecionar uma playlist em uma interface web e iniciar sua reprodução em um dispositivo Spotify Connect, com autenticação segura, tratamento de falhas e configuração local.

1. Solução adotada

Foi desenvolvida uma aplicação web simples utilizando Python, Flask e a Spotify Web API. O usuário acessa uma página com três opções de playlists previamente configuradas e sem conteúdo explícito. Ao escolher uma opção e clicar em “Reproduzir playlist”, o formulário envia a seleção ao backend.

O backend identifica a playlist selecionada, consulta os dispositivos Spotify Connect disponíveis, direciona a reprodução para o dispositivo escolhido ou ativo e solicita ao Spotify o início da playlist. A autenticação é realizada previamente via OAuth 2.0 e o refresh token é armazenado localmente para evitar novo login durante a demonstração.

2. Tecnologias utilizadas

- Python 3, Flask, Requests e python-dotenv - backend, integração REST e configuração.
- Spotify Web API - autenticação, consulta de dispositivos e controle de reprodução.
- OAuth 2.0 Authorization Code - autorização segura da conta Spotify.
- HTML/CSS - formulário para seleção da playlist.
- Spotify Connect - dispositivo final em que a playlist será reproduzida.

3. Opções de playlists

Para garantir o requisito de ausência de palavrões ou conteúdo explícito, as três playlists devem ser criadas e revisadas pelo grupo antes da apresentação. O projeto já vem configurado para receber três URIs de playlists no arquivo .env:

- Focus Instrumental
- Lo-fi Clean
- Acoustic Clean

4. Fluxo da automação



Figura 1 - Fluxo de seleção e reprodução da playlist.

1. O usuário acessa o formulário web.
2. Seleciona uma das três playlists disponíveis.
3. O formulário envia a seleção para o endpoint POST /play.
4. O backend identifica a URI correspondente à playlist selecionada.
5. O sistema consulta os dispositivos disponíveis na conta Spotify.
6. O backend transfere o playback para o dispositivo selecionado ou ativo.
7. A Spotify Web API recebe a URI da playlist e inicia a reprodução.
8. A tela retorna uma confirmação de sucesso para o usuário.

5. Endpoints da Spotify Web API utilizados

Endpoint	Método	Função
/v1/me/player/devices	GET	Obtém os dispositivos disponíveis.
/v1/me/player	PUT	Transfere o playback para o dispositivo desejado.
/v1/me/player/play	PUT	Inicia a reprodução da playlist selecionada.

6. Permissões solicitadas

- user-read-playback-state - necessária para consultar os dispositivos Spotify Connect.
- user-modify-playback-state - necessária para transferir e iniciar a reprodução.

7. Preparação para a validação em aula

1. Utilizar uma conta Spotify Premium e manter a assinatura ativa.
2. Criar o aplicativo no Spotify Developer Dashboard e cadastrar exatamente o redirect URI http://127.0.0.1:5000/callback.
3. Configurar Client ID, Client Secret e as três URIs das playlists no arquivo .env.
4. Executar a aplicação com python app.py antes do início da validação.
5. Abrir o Spotify Desktop ou o aplicativo móvel na mesma conta.
6. Reproduzir manualmente qualquer música por alguns segundos para garantir que o dispositivo apareça no Spotify Connect.
7. Deixar a página do formulário aberta e a autenticação concluída antes da apresentação.

8. Tratamento de falhas

Situação	Ação prevista
Token expirado	O backend utiliza o refresh token para obter automaticamente um novo access token.
Nenhum dispositivo disponível	A interface orienta a abrir o Spotify e ativar um dispositivo.
Erro 403	Verificar Spotify Premium, usuário autorizado no app e permissões OAuth.
Erro 429	Respeitar o tempo de Retry-After retornado pela API.
Erros 500, 502, 503 ou 504	Repetir automaticamente a chamada de reprodução em até três tentativas.

9. Estrutura do projeto

- app.py - lógica da aplicação, OAuth e integração com o Spotify.
- templates/index.html - formulário de escolha da playlist.
- templates/result.html - confirmação do resultado.
- static/style.css - estilização da interface.
- .env.example - modelo das variáveis de ambiente.
- requirements.txt - dependências Python.
- README.md - instruções completas de configuração e execução.

10. Atendimento aos requisitos do CP2

- Linguagem - automação e backend desenvolvidos em Python.
- Integração externa - consumo da Spotify Web API REST com Requests.
- Qualidade - responsabilidades separadas, tratamento de falhas e teste automatizado.
- Dependências - bibliotecas e versões registradas em requirements.txt.

11. Conclusão

A solução atende aos requisitos do CP2 ao empregar Python, integração REST, dependências documentadas e uma interface objetiva para controlar playlists no Spotify. O fluxo foi mantido simples e resiliente para facilitar a demonstração, a manutenção e a análise técnica.

Referências técnicas

https://developer.spotify.com/documentation/web-api

https://developer.spotify.com/documentation/web-api/reference/get-a-users-available-devices

https://developer.spotify.com/documentation/web-api/concepts/redirect_uri

https://developer.spotify.com/documentation/web-api/concepts/quota-modes